

JSON

JSON: **JavaScript Object Notation**.

JSON is a syntax for storing and exchanging data.

JSON is text, written with JavaScript object notation.

JSON or JavaScript Object Notation

- A lightweight text-based open standard designed for human-readable data interchange.
- The JSON format was originally specified by Douglas Crockford, and is described in RFC 4627.
- The official Internet media type for JSON is application/json.
- The JSON filename extension is .json.

JSON or JavaScript Object Notation

- JSON stands for JavaScript Object Notation.
- The format was specified by Douglas Crockford.
- It was designed for human-readable data interchange.
- It has been extended from the JavaScript scripting language.
- The filename extension is **.json**.
- JSON Internet Media type is **application/json**.
- The Uniform Type Identifier is public.json.

Uses of JSON

- It is used while writing JavaScript based applications that includes browser extensions and websites.
- JSON format is used for serializing and transmitting structured data over network connection.
- It is primarily used to transmit data between a server and web applications.
- Web services and APIs use JSON format to provide public data.
- It can be used with modern programming languages.

Characteristics of JSON

- JSON is easy to read and write.
- It is a lightweight text-based interchange format.
- JSON is language independent.

Simple Example in JSON

```
{
  "book": [
    {
      "id": "01",
      "language": "Java",
      "edition": "third",
      "author": "Herbert Schildt"
    },
    {
      "id": "07",
      "language": "C++",
      "edition": "second",
      "author": "E.Balagurusamy"
    }
  ]
}
```

EX jsonEx01.html

- Run file jsonEx01.html
- Result

JSON with JavaScript example

Language = Java

Author = herbert schildt

Language = C++

Author = E-Balagurusamy

C++ programming language can be studied from book written by E-Balagurusamy

```
<html>
  <head>
    <title>JSON example</title>

    <script language = "javascript" >

      var object1 = { "language" : "Java", "author" : "herbert schildt" };
      document.write("<h1>JSON with JavaScript example</h1>");
      document.write("<br>");
      document.write("<h3>Language = " + object1.language+"</h3>");
      document.write("<h3>Author = " + object1.author+"</h3>");

      var object2 = { "language" : "C++", "author" : "E-Balagurusamy" };
      document.write("<br>");
      document.write("<h3>Language = " + object2.language+"</h3>");
      document.write("<h3>Author = " + object2.author+"</h3>");

      document.write("<hr />");
      document.write(object2.language + " programming language can be
        studied " + "from book written by " + object2.author);
      document.write("<hr />");

    </script>
  </head>

  <body>
  </body>
</html>
```

Basic syntax of JSON

JSON syntax is basically considered as a subset of JavaScript syntax.

- Data is represented in name/value pairs.
- Curly braces (..{ }..) hold objects and each name is followed by ':'(colon), the name/value pairs are separated by , (comma).
- Square brackets hold arrays and values are separated by ,(comma).

```
{  
  "book": [  
    {  
      "id": "01",  
      "language": "Java",  
      "edition": "third",  
      "author": "Herbert Schildt"  
    },  
    {  
      "id": "07",  
      "language": "C++",  
      "edition": "second",  
      "author": "E.Balagurusamy"  
    }  
  ]  
}
```

Valid Data Types

- a string
- a number
- an object (JSON object)
- an array
- a boolean
- *null*

JSON Strings

- Strings in JSON must be written in double quotes.

Example

```
{ "name": "John" }
```

JSON Numbers

- Numbers in JSON must be an integer or a floating point.

Example

```
{ "age":30 }
```

JSON Objects

- Values in JSON can be objects.

Example

```
{  
  "employee":{ "name":"John", "age":30, "city":"New York" }  
}
```

JSON Arrays

- Values in JSON can be arrays.

Example

```
{  
  "employees" : [ "John", "Anna", "Peter" ]  
}
```

JSON Booleans

- Values in JSON can be true/false.

Example

```
{ "sale" : true }
```

JSON null

- Values in JSON can be null.

Example

```
{ "middlename" : null }
```

JSON.parse()

- A common use of JSON is to exchange data to/from a web server.
- When receiving data from a web server, the data is always a string.
- Parse the data with JSON.parse(), and the data becomes a JavaScript object.

Example - Parsing JSON : Imagine we received this text from a web server:

```
{ "name":"John", "age":30, "city":"New York" }
```

Use the JavaScript function JSON.parse() to convert text into a JavaScript object:

```
var obj = JSON.parse( '{ "name":"John", "age":30, "city":"New York" }' );
```

Use the JavaScript object in your page

jsonEx02.html

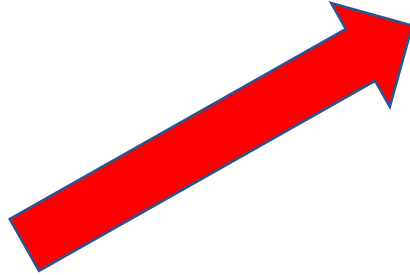
```
<!DOCTYPE html>
<html>
<body>

<h2>Create Object from JSON String</h2>

<p id="demo"></p>

<script>
var txt = '{"name":"John", "age":30, "city":"New York"}'
var obj = JSON.parse(txt);
document.getElementById("demo").innerHTML = obj.name + ", " + obj.age;
</script>

</body>
</html>
```



Create Object from JSON String

John, 30

JSON From the Server

- You can request JSON from the server by using an AJAX request As long as the response from the server is written in JSON format, you can parse the string into a JavaScript object.

jsonEx03.html

```
<!DOCTYPE html>
<html>
<body>

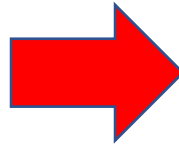
<h2>Use the XMLHttpRequest to get the content of a file.</h2>
<p>The content is written in JSON format, and can easily be converted into a JavaScript
object.</p>

<p id="demo"></p>

<script>
var xmlhttp = new XMLHttpRequest();
xmlhttp.onreadystatechange = function() {
  if (this.readyState == 4 && this.status == 200) {
    var myObj = JSON.parse(this.responseText);
    document.getElementById("demo").innerHTML = myObj.name;
  }
};
xmlhttp.open("GET", "json_demo.txt", true);
xmlhttp.send();
</script>

<p>Take a look at <a href="json_demo.txt" target="_blank">json_demo.txt</a></p>

</body>
</html>
```



Use the XMLHttpRequest to get the content of a file.

The content is written in JSON format, and can easily be converted into a JavaScript object.

John

Take a look at json_demo.txt

Array as JSON

- When using the `JSON.parse()` on a JSON derived from an array, the method will return a JavaScript array, instead of a JavaScript object.

jsonEx04.html

```
<!DOCTYPE html>
<html>
<body>

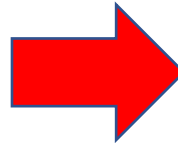
<h2>Content as Array.</h2>
<p>Content written as an JSON array will be converted into a JavaScript array.</p>

<p id="demo"></p>

<script>
var xmlhttp = new XMLHttpRequest();
xmlhttp.onreadystatechange = function() {
  if (this.readyState == 4 && this.status == 200) {
    var myArr = JSON.parse(this.responseText);
    document.getElementById("demo").innerHTML = myArr[0];
  }
};
xmlhttp.open("GET", "json_demo_array.txt", true);
xmlhttp.send();
</script>

<p>Take a look at <a href="json_demo_array.txt" target="_blank">json_demo_array.txt</a>
</p>

</body>
</html>
```



Content as Array.

Content written as an JSON array will be converted into a JavaScript array.

Ford

Take a look at json_demo_array.txt

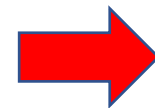
Exceptions

Parsing Dates

- Date objects are not allowed in JSON.
- If you need to include a date, write it as a string.
- You can convert it back into a date object later:
- Ex. Convert a string into a date:

jsonEx05.html

```
var text = '{ "name":"John", "birth":"1986-12-14", "city":"New York"}';  
var obj = JSON.parse(text);  
obj.birth = new Date(obj.birth);  
  
document.getElementById("demo").innerHTML = obj.name + ", " + obj.birth;
```



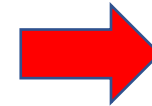
Convert a string into a date object.

John, Sun Dec 14 1986 07:00:00 GMT+0700 (Indochina Time)

- Convert a string into a date, using the reviver function:

```
var text = '{ "name":"John", "birth":"1986-12-14", "city":"New York"}';
var obj = JSON.parse(text, function (key, value) {
    if (key == "birth") {
        return new Date(value);
    } else {
        return value;
    }
});
```

jsonEx06.html



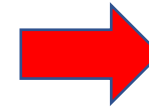
Convert a string into a date object.

John, Sun Dec 14 1986 07:00:00 GMT+0700 (Indochina Time)

```
document.getElementById("demo").innerHTML = obj.name + ", " + obj.birth;
```

Parsing Functions

- Functions are not allowed in JSON.
- If you need to include a function, write it as a string.
- You can convert it back into a function later:
- Ex. Convert a string into a function:



Convert a string into a function.

John, 30

```
var text = '{ "name":"John", "age":"function () {return 30;}", "city":"New York"}';
```

```
var obj = JSON.parse(text);
```

```
obj.age = eval("(" + obj.age + ")");    jsonEx07.html
```

```
document.getElementById("demo").innerHTML = obj.name + ", " + obj.age();
```

JSON.stringify()

- A common use of JSON is to exchange data to/from a web server.
- When sending data to a web server, the data has to be a string.
- Convert a JavaScript object into a string with JSON.stringify().

Stringify a JavaScript Object

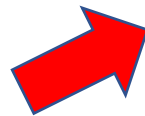
this object in JavaScript:

```
var obj = { name: "John", age: 30, city: "New York" };
```

Use the JavaScript function `JSON.stringify()` to convert it into a string.

```
var myJSON = JSON.stringify(obj);
```

`myJSON` is now a string, and ready to be sent to a server.



Create JSON string from a JavaScript object.

```
{"name":"John","age":30,"city":"New York"}
```

```
var obj = { name: "John", age: 30, city: "New York" };
```

```
var myJSON = JSON.stringify(obj);
```

```
document.getElementById("demo").innerHTML = myJSON;
```

jsonEx08.html

Stringify a JavaScript Array

this array in JavaScript

- `var arr = [ "John", "Peter", "Sally", "Jane" ];`

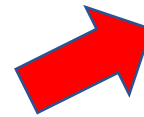
Use the JavaScript function `JSON.stringify()` to convert it into a string.

- `var myJSON = JSON.stringify(arr);`

```
var arr = [ "John", "Peter", "Sally", "Jane" ];  
var myJSON = JSON.stringify(arr);  
document.getElementById("demo").innerHTML = myJSON;
```

Create JSON string from a JavaScript array.

`["John","Peter","Sally","Jane"]`



jsonEx09.html

Exceptions

Stringify Dates

- In JSON, date objects are not allowed. The `JSON.stringify()` function will convert any dates into strings.
- You can convert the string back into a date object at the receiver.

```
var obj = { name: "John", today: new Date(), city : "New York" };  
var myJSON = JSON.stringify(obj);
```

```
document.getElementById("demo").innerHTML = myJSON;
```

jsonEx10.html



JSON.stringify will convert any date objects into strings.

```
{"name":"John","today":"2020-01-16T10:57:24.583Z","city":"New York"}
```

Stringify Functions

- In JSON, functions are not allowed as object values.
- The `JSON.stringify()` function will remove any functions from a JavaScript object, both the key and the value:

```
var obj = { name: "John", age: function () {return 30;}, city: "New York"};  
var myJSON = JSON.stringify(obj);
```

```
document.getElementById("demo").innerHTML = myJSON;    jsonEx11.html
```

- convert your functions into strings before running the `JSON.stringify()` function.

```
var obj = { name: "John", age: function () {return 30;}, city: "New York" };  
obj.age = obj.age.toString();  
var myJSON = JSON.stringify(obj);
```

jsonEx12.html

```
document.getElementById("demo").innerHTML = myJSON;
```

JSON Objects

Object Syntax

- JSON objects are surrounded by curly braces {}.
- JSON objects are written in key/value pairs.
- Keys must be strings, and values must be a valid JSON data type (string, number, object, array, boolean or null).
- Keys and values are separated by a colon.
- Each key/value pair is separated by a comma.
- Ex. { "name":"John", "age":30, "car":null }

Accessing Object Values

```
myObj = { "name": "John", "age": 30, "car": null };  
x = myObj.name;
```

jsonEx13.html

You can also access the object values by using bracket ([]) notation:

```
myObj = { "name": "John", "age": 30, "car": null };  
x = myObj["name"];
```

jsonEx14.html

Looping an Object

```
myObj = { "name":"John", "age":30, "car":null };  
for (x in myObj) {  
    document.getElementById("demo").innerHTML += x;  
}
```

jsonEx15.html

```
myObj = { "name":"John", "age":30, "car":null };  
for (x in myObj) {  
    document.getElementById("demo").innerHTML += myObj[x];  
}
```

jsonEx16.html

Nested JSON Objects

- You can access nested JSON objects by using the dot notation or bracket notation:

```
myObj = {  
  "name": "John",  
  "age": 30,  
  "cars": {  
    "car1": "Ford",  
    "car2": "BMW",  
    "car3": "Fiat"  
  }  
}
```

jsonEx17.html

Modify Values

```
myObj.cars.car2 = "Mercedes";
```

jsonEx18.html

```
myObj.cars["car2"] = "Mercedes";
```

jsonEx19.html

Delete Object Properties

```
delete myObj.cars.car2;
```

jsonEx20.html

JSON Arrays

- Ex. ["Ford", "BMW", "Fiat"]
- Arrays in JSON are almost the same as arrays in JavaScript.
- In JSON, array values must be of type string, number, object, array, boolean or *null*.

Arrays in JSON Objects

```
{  
  "name": "John",  
  "age": 30,  
  "cars": [ "Ford", "BMW", "Fiat" ]  
}
```

jsonEx21.html

Accessing Array Values

```
x = myObj.cars[0];
```

Looping Through an Array

```
for (i in myObj.cars) {  
    x += myObj.cars[i];  
}
```

jsonEx22.html

```
for (i = 0; i < myObj.cars.length; i++) {  
    x += myObj.cars[i];  
}
```

jsonEx23.html

Nested Arrays in JSON Objects

```
myObj = {  
  "name": "John",  
  "age": 30,  
  "cars": [  
    { "name": "Ford", "models": [ "Fiesta", "Focus", "Mustang" ] },  
    { "name": "BMW", "models": [ "320", "X3", "X5" ] },  
    { "name": "Fiat", "models": [ "500", "Panda" ] }  
  ]  
}
```

jsonEx24.html

```
for (i in myObj.cars) {  
  x += "<h1>" + myObj.cars[i].name + "</h1>";  
  for (j in myObj.cars[i].models) {  
    x += myObj.cars[i].models[j];  
  }  
}
```

Modify Array Values

- `myObj.cars[1] = "Mercedes";`

`jsonEx25.html`

Delete Array Items

- `delete myObj.cars[1];`

[jsonEx26.html](#)

JSON HTML

- HTML Table [jsonEx27.html](#)
- Dynamic HTML Table [jsonEx28.html](#)
- HTML Drop Down List [jsonEx29.html](#)